



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET
KATEDRA ZA RAČUNARSTVO



Seminarski rad

Monitoring i observability u Microsoft SQL Serveru

Student:

Jovana Stepanović br. ind. 1831

Mentor:

Prof. Aleksandar Stanimirović

Sadržaj

1. Uvod.....	3
2. Teorijske osnove monitoringa i observabilnosti	4
3. DMV-ovi i wait statistics.....	5
3.1. Dynamic Management Views i Dynamic Management Functions	5
3.2. Wait statistics.....	6
4. Extended Events	9
5. Query Store	13
6. Proceduralni deo, zašto se monitoring pakuje u proceduru?	16
7. Blocking i deadlock.....	21
8. Zaključak	26
9. Literatura	27

1. Uvod

Funkcionisanje sistema za upravljanje bazama podataka u produkcionim uslovima retko je statično. Opterećenje varira tokom dana, upiti čije je ponašanje juče bilo predvidivo mogu danas postati spori zbog promene plana izvršavanja, a transakcije koje se preklapaju mogu dovesti do blokada koje korisnici osećaju kao "sistem je stao". Ako administrator nema uvid u to šta se trenutno događa unutar servera, dijagnostika ovakvih problema svodi se na pogađanje, a rešavanje na pokušaj i grešku u produkciji, što u bankarskim i finansijskim sistemima nije prihvatljivo. Iz ove potrebe proizašla je disciplina monitoringa i observabilnosti baza podataka, skup alata i praksa koji administratoru omogućavaju da u svakom trenutku odgovori na pitanja: koje sesije su trenutno aktivne i na šta čekaju, gde je server ukupno izgubio vreme, koji upiti su se ponašali najgore i da li konkurentne transakcije stvaraju blokade koje ugrožavaju dostupnost sistema. Razlika između monitoringa i observabilnosti, kao i sama terminologija, biće detaljnije objašnjena u narednom poglavlju, ali već na ovom mestu treba istaći da nije reč o jednokratnoj aktivnosti, već o kontinuiranom procesu koji se ugrađuje u svakodnevni rad sa bazom. Microsoft SQL Server pruža bogat skup mehanizama za ovu svrhu, od Dynamic Management Views koji izlažu trenutno stanje servera, preko Extended Events kao naslednika starijeg SQL Trace mehanizma za snimanje događaja, do Query Store-a koji čuva istoriju izvršavanja upita i omogućava poređenje performansi kroz vreme. Pored ovih ugrađenih alata, u praksi se monitoring logika često automatizuje kroz stored procedure koje se pozivaju periodično, čime se administratoru oslobađa potreba za ručnim pokretanjem upita svaki put kada želi da proveriti stanje sistema.

U ovom radu biće prikazane teorijske osnove monitoringa i observabilnosti, a zatim, kroz primer baze podataka BankDB, praktična primena DMV-ova, Extended Events sesija, Query Store-a, kao i implementacija proceduralnog mehanizma za automatizovanu proveru stanja baze. Posebna pažnja posvećena je detekciji blokada i deadlock-ova, situacijama koje u bankarskim sistemima sa čestim konkurentnim transakcijama predstavljaju jedan od najčešćih praktičnih problema.

2. Teorijske osnove monitoringa i observabilnosti

Termini monitoring i observabilnost često se koriste kao sinonimi, ali između njih postoji važna razlika koju je korisno razjasniti na početku. Monitoring predstavlja praćenje predefinisane skupa metrika i signalizaciju kada neka od njih izađe iz očekivanog opsega, primer je alarm koji se aktivira kada CPU iskorišćenje servera prelazi 90% ili kada disk ostane bez slobodnog prostora. Monitoring odgovara na pitanje da li je nešto u redu, na osnovu unapred poznatih indikatora. Observabilnost ide korak dalje i odgovara na pitanje zašto sistem trenutno radi onako kako radi, čak i u situacijama koje administrator nije unapred predvideo. Dok monitoring zahteva da se unapred definiše šta se prati, observabilnost se zasniva na dovoljno detaljnim podacima da omoguće istraživanje problema koji ranije nije postojao u sistemu alarma. Ova razlika oslanja se na tri stuba koja se najčešće navode kao temelj observabilnosti. Metrike su numeričke vrednosti agregirane kroz vreme, na primer prosečan broj upita u sekundi ili procenat iskorišćenosti memorije, i pogodne su za praćenje trendova i postavljanje alarma. Logovi predstavljaju vremenski uređene zapise o diskretnim događajima, na primer poruku da je određena transakcija odbijena ili da je restartovan servis, i korisni su za rekonstrukciju tačnog sleda događaja koji je doveo do problema. Tragovi (traces) prate put jednog zahteva kroz sistem, što je u distribuiranim arhitekturama (mikroservisi, više baza) ključno za pronalaženje gde u lancu poziva nastaje kašnjenje, dok u kontekstu jednog SQL Server instance trace najčešće odgovara detaljnom snimku izvršavanja jednog upita, od momenta kada stigne na server do momenta kada se vrati rezultat.

U kontekstu sistema za upravljanje bazama podataka, ova tri stuba prevode se u konkretne kategorije praćenja. Administrator u produkcionom okruženju tipično prati resurse servera (CPU, memoriju, disk I/O), aktivne sesije i čekanja koja usporavaju rad, performanse pojedinačnih upita, konkurentne transakcije koje mogu dovesti do blokada, kao i rast baze i njenih indeksa radi planiranja kapaciteta. Svaka od ovih kategorija ima svoj skup ugrađenih mehanizama u SQL Serveru. U ovom radu fokus je na trima od navedenih kategorija: aktivnim sesijama i akumuliranim čekanjima, performansama pojedinačnih upita kroz vreme, i konkurentnim transakcijama sa fokusom na blokade i deadlock-ove. Monitoring resursa servera i praćenje rasta baze zahtevaju sopstvenu, širu obradu i ostavljaju se van obima ovog rada.

Microsoft SQL Server pokriva navedene kategorije kroz kombinaciju ugrađenih mehanizama, koji će biti detaljno predstavljeni u narednim poglavljima: Dynamic Management Views i wait statistics za aktivne sesije i čekanja, Extended Events za snimanje detaljnih događaja, Query Store za performanse upita kroz vreme, i kombinacija ovih alata, upakovana u proceduralni mehanizam, za kontinuirano i automatizovano praćenje stanja baze u produkcionom okruženju. Kroz rad se kao demonstracioni primer koristi baza BankDB. Baza sadrži tabele Accounts (računi klijenata), Transactions (transakcije nad računima) i Employees (podaci o zaposlenima)...

3. DMV-ovi i wait statistics

3.1. Dynamic Management Views i Dynamic Management Functions

Dynamic Management Views (DMV) i Dynamic Management Functions (DMF) predstavljaju skup sistemskih pogleda i funkcija koje SQL Server izlaže kako bi administratoru omogućio uvid u trenutno interno stanje servera i baza podataka, bez potrebe za eksternim alatima. Za razliku od običnih tabela koje čuvaju trajne podatke, DMV-ovi predstavljaju momentalni snimak (snapshot) onoga što se događa u trenutku kada se upit izvrši, podaci u njima se kontinuirano ažuriraju dok server radi, i najveći deo njih se resetuje pri restartu servisa.

DMV-ovi su organizovani u šemu `sys.dm_*`, a razlikuju se po prefiksu koji ukazuje na kategoriju informacija koje izlažu. Najčešće korišćene kategorije su:

- **exec** — informacije o izvršavanju upita i sesija (npr. `sys.dm_exec_requests`, `sys.dm_exec_sessions`, `sys.dm_exec_query_stats`)
- **os** — informacije na nivou operativnog sistema kako ih SQL Server vidi (npr. `sys.dm_os_wait_stats`, `sys.dm_os_sys_memory`)
- **tran** — informacije o transakcijama i zaključavanju (npr. `sys.dm_tran_locks`)
- **db** — informacije specifične za bazu podataka (npr. `sys.dm_db_index_usage_stats`)

Važna razlika u odnosu na obične upite jeste da DMV-ovi zahtevaju posebnu permisiju, najčešće `VIEW SERVER STATE` za poglede na nivou servera i `VIEW DATABASE STATE` za poglede na nivou baze, što znači da pristup ovim podacima sam po sebi predstavlja bezbednosno pitanje. Korisnik koji ima ovu permisiju može da vidi tekst upita drugih korisnika, što u bankarskim sistemima sa osetljivim podacima treba dodeliti isključivo administratorima.

Da bismo demonstrirali osnovnu upotrebu DMV-ova, posmatramo trenutno aktivne zahteve na serveru kroz pogled `sys.dm_exec_requests`, povezan sa `sys.dm_exec_sessions` radi dobijanja informacija o korisniku koji je zahtev pokrenuo. Da bi se uhvatila sesija u aktivnom stanju čekanja, u drugom query tabu pokrenuta je naredba `WAITFOR DELAY '00:00:15'`, dok je u prvom tabu, u tom istom intervalu, izvršen upit nad `sys.dm_exec_requests`.

```

1  USE BankDB;
2  GO
3
4  -- sta se trenutno izvršava na serveru (DMV: sys.dm_exec_requests)
5  SELECT
6      r.session_id,
7      r.status,
8      r.command,
9      r.cpu_time,
10     r.total_elapsed_time,
11     r.wait_type,
12     s.login_name,
13     s.host_name
14 FROM sys.dm_exec_requests r
15 JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
16 WHERE r.session_id != @@SPID; -- isključujemo svoju sopstvenu sesiju
17 GO
18

```

100 % No issues found

session_id	status	command	cpu_time	total_elapsed_time	wait_type	login_name	host_name
99	103	background	XTP_THREAD_POOL	0	1298545606	DISPATCHER_QUEUE_SEMAPHORE	sa
100	104	background	XTP_THREAD_POOL	0	1298545606	DISPATCHER_QUEUE_SEMAPHORE	sa
101	105	background	XTP_THREAD_POOL	15	1298545606	DISPATCHER_QUEUE_SEMAPHORE	sa
102	106	background	XTP_THREAD_POOL	0	1298545557	DISPATCHER_QUEUE_SEMAPHORE	sa
103	107	background	XTP_THREAD_POOL	0	1298545557	DISPATCHER_QUEUE_SEMAPHORE	sa
104	108	background	XTP_THREAD_POOL	0	1298545557	DISPATCHER_QUEUE_SEMAPHORE	sa
105	109	background	XTP_THREAD_POOL	0	1298545557	DISPATCHER_QUEUE_SEMAPHORE	sa
106	110	background	XTP_THREAD_POOL	0	1298545557	DISPATCHER_QUEUE_SEMAPHORE	sa
107	111	background	XTP_THREAD_POOL	0	1298545557	DISPATCHER_QUEUE_SEMAPHORE	sa
108	112	background	XTP_THREAD_POOL	0	1298545557	DISPATCHER_QUEUE_SEMAPHORE	sa
109	113	background	XTP_THREAD_POOL	0	1298545557	DISPATCHER_QUEUE_SEMAPHORE	sa
110	114	sleeping	TASK MANAGER	0	0	NULL	sa
111	117	sleeping	TASK MANAGER	0	0	NULL	sa
112	118	sleeping	TASK MANAGER	0	0	NULL	sa
113	119	sleeping	TASK MANAGER	0	0	NULL	sa
114	120	sleeping	TASK MANAGER	0	0	NULL	sa
115	121	sleeping	TASK MANAGER	0	0	NULL	sa
116	122	suspended	WAITFOR	0	2543	WAITFOR	DESKTOP-AM7N4IV\Optimus
							DESKTOP-AM7N4IV

Slika 3.1.1. — Među redovima koji prikazuju interne background procese servera (sa wait_type DISPATCHER_QUEUE_SEMAPHORE i XTP_THREAD_POOL, koji su uobičajeni i ne zahtevaju pažnju administratora), izdvaja se sesija sa session_id 122, čiji je status suspended, što znači da sesija aktivno čeka i ne koristi CPU u tom trenutku. Kolona command potvrđuje da se izvršava naredba WAITFOR, a wait_type takođe prikazuje vrednost WAITFOR, čime se potvrđuje direktna veza između naredbe koja se izvršava i razloga čekanja. Kolona total_elapsed_time pokazuje da je sesija u tom trenutku bila aktivna 2543 milisekunde, dok login_name i host_name identifikuju korisnika i radnu stanicu sa koje je upit pokrenut.

Ovaj primer ilustruje osnovnu vrednost sys.dm_exec_requests: administrator u svakom trenutku može da vidi ne samo da je server "spor", već tačno koja sesija, koji korisnik i koja naredba je razlog za to, bez potrebe da čeka da korisnik prijavi problem.

3.2. Wait statistics

Dok sys.dm_exec_requests (slika 3.1.1) prikazuje trenutni momenat, ko sad nešto radi i na šta sad čeka, wait statistics gledaju dugoročnu sliku, na šta je server ukupno, od poslednjeg restarta ili od poslednjeg resetovanja brojača, najviše trošio vreme čekajući. Kada sesija ne može odmah da nastavi sa radom, jer čeka CPU, disk, lock ili mrežu, taj period se beleži kao wait, uz tip čekanja (wait_type) koji objašnjava zašto se čekalo. SQL Server akumulira ove podatke u sys.dm_os_wait_stats, gde se za svaki tip čekanja čuva ukupan broj puta kada se desilo i ukupno vreme provedeno čekajući.

Praktična vrednost ovoga je da wait statistics ne govore samo da je server spor, već usmeravaju administratora gde da traži uzrok. Dominantan PAGEIOLATCH ukazuje na spor disk ili nedostatak memorije za keširanje, dok dominantan LCK_M_X ukazuje na probleme sa blokadama, što je detaljno obrađeno u poglavlju 7.

Važno je napomenuti da sirova lista wait_type vrednosti sadrži veliki broj sistemskih, internih čekanja koja su uvek prisutna dok server radi i ne nose dijagnostičku vrednost, poput pratećih taskova za raspoređivanje CPU-a ili periodičnog čišćenja keša. Zbog toga se u praksi ova lista filtrira, izbacujući poznate benigne tipove, kako bi preostali rezultat ukazivao isključivo na potencijalno relevantna čekanja.

Nakon resetovanja brojača statistike (DBCC SQLPERF), kreiranja simulirane sesije i izvršavanja filtriranog upita nad sys.dm_os_wait_stats, dobijen je rezultat prikazan na slici 3.2.1. Upit prikazuje i kolone signal_wait_time_ms i resource_wait_time_ms, koje pomažu da se bolje razume zbog čega je došlo do čekanja. Signal wait predstavlja vreme koje je sesija provela čekajući da dobije CPU nakon što je potreban resurs već bio dostupan. Sa druge strane, resource wait označava vreme čekanja na sam resurs, kao što su disk, memorija ili zaključavanje (lock). Ova razlika je značajna prilikom analize performansi, jer veći signal wait obično ukazuje na opterećenje procesora, dok veći resource wait pokazuje da je problem u određenom sistemskom resursu koji predstavlja usko grlo.

```

1  SELECT TOP 10
2      wait_type,
3      waiting_tasks_count,
4      wait_time_ms,
5      signal_wait_time_ms,
6      wait_time_ms - signal_wait_time_ms AS resource_wait_time_ms
7  FROM sys.dm_os_wait_stats
8  WHERE wait_type NOT IN (
9      'CLR_SEMAPHORE', 'LAZYWRITER_SLEEP', 'RESOURCE_QUEUE',
10     'SLEEP_TASK', 'SLEEP_SYSTEMTASK', 'SQLTRACE_BUFFER_FLUSH',
11     'WAITFOR', 'LOGMGR_QUEUE', 'CHECKPOINT_QUEUE',
12     'REQUEST_FOR_DEADLOCK_SEARCH', 'XE_TIMER_EVENT', 'BROKER_TO_FLUSH',
13     'BROKER_TASK_STOP', 'CLR_MANUAL_EVENT', 'CLR_AUTO_EVENT',
14     'DISPATCHER_QUEUE_SEMAPHORE', 'FT_ISTS_SCHEDULER_IDLE_WAIT',
15     'XE_DISPATCHER_WAIT', 'XE_DISPATCHER_JOIN', 'SQLTRACE_INCREMENTAL_FLUSH_SLEEP',
16     'BROKER_EVENTHANDLER', 'TRACEWRITE', 'BROKER_RECEIVE_WAITFOR',
17     'SOS_WORK_DISPATCHER', 'PWAIT_DIRECTLOGCONSUMER_GETNEXT',
18     'DIRTY_PAGE_POLL', 'HADR_FILESTREAM_IOMGR_IOCOMPLETION'
19 )
20 ORDER BY wait_time_ms DESC;
21 GO

```

100 % ✔ No issues found

	wait_type	waiting_tasks_count	wait_time_ms	signal_wait_time_ms	resource_wait_time_ms
1	QDS_PERSIST_TASK_MAIN_LOOP_SLEEP	2	120015	0	120015
2	PREEMPTIVE_OS_QUERYREGISTRY	83	4	0	4
3	PREEMPTIVE_OS_WRITEFILE	1	2	0	2
4	WRITELOG	2	1	0	1
5	MEMORY_ALLOCATION_EXT	1135	1	0	1
6	WAIT_SLEEP_LOWMEM_GUARD	0	0	0	0
7	RESERVED_MEMORY_ALLOCATION_EXT	12	0	0	0
8	IO_QUEUE_LIMIT	0	0	0	0
9	SESSION_WAIT_STATS_CHILDREN	0	0	0	0
10	PREEMPTIVE_XE_DISPATCHER	0	0	0	0

Slika 3.2.1. — Rezultat pokazuje da na lokalnom, neopterećenom serveru ni jedan wait tip ne prelazi vrednosti koje bi ukazivale na stvarni problem. Najveće akumulirano vreme pripada tipu QDS_PERSIST_TASK_MAIN_LOOP_SLEEP, internom background tasku koji periodično upisuje podatke prikupljene kroz Query Store mehanizam (detaljnije obrađen u poglavlju 5), a ne korisničkoj aktivnosti. Preostali tipovi čekanja (PREEMPTIVE_OS_QUERYREGISTRY, WRITELOG, MEMORY_ALLOCATION_EXT) imaju zanemarljive vrednosti, ispod 100 milisekundi, što potvrđuje da server trenutno ne trpi pritisak na disk, CPU niti memoriju. Ovaj rezultat pokazuje da cilj health check analize nije uvek pronalaženje problema, već i potvrda, na osnovu objektivnih pokazatelja, da sistem radi stabilno i u skladu sa očekivanjima.

4. Extended Events

Extended Events (XE) predstavlja moderniji mehanizam za snimanje detaljnih događaja na SQL Serveru, koji je zamenio stariji SQL Trace/Profiler. Razlog zamene je performans, Profiler je radio kao eksterni "prisluškivač" koji značajno opterećuje server, dok je Extended Events ugrađen direktno u SQL Server engine i ima znatno manji uticaj na performanse, čak i kada se koristi u produkciji.

Arhitektura Extended Events sastoji se od četiri osnovna pojma. Session je kontejner koji povezuje sve ostale delove, kreira se, konfiguriše i pokreće/zaustavlja kao celina. Event je konkretan događaj koji se prati, na primer "upit je završen izvršavanje" (sql_statement_completed), SQL Server ima stotine ugrađenih event-a, a administrator bira samo one relevantne za konkretan scenario. Action je dodatna informacija koja se priključuje uz event kada se desi, na primer tekst upita (sql_text) ili ime baze (database_name), event sam po sebi nosi osnovne podatke, a action-i ga obogaćuju kontekstom. Target je odredište gde se podaci čuvaju, najčešće ring_buffer (memorija, gubi se pri restartu, pogodno za kratkotrajnu analizu) ili event_file (.xel fajl na disku, trajno).

Ključna prednost ovog modela je filtriranje na nivou predikata, administrator definiše da se event hvata samo ako ispunjava uslov, na primer samo ako trajanje upita prelazi određeni prag. Time se broj uhvaćenih događaja drastično smanjuje, što čini analizu preglednijom, jer se hvataju samo upiti koji stvarno zaslužuju pažnju.

Važno je napomenuti da SQL Server po default-u već pokreće jednu ugrađenu Extended Events sesiju pod imenom system_health, koja hvata kritične sistemske događaje bez ikakve intervencije administratora, između ostalog deadlock-ove (kroz event xml_deadlock_report), greške sa severity nivoom većim od 16, i wait-ove koji traju duže od 15 sekundi. U produkcionom okruženju administrator obično prvo konsultuje sadržaj system_health sesije kada se desi problem, a tek zatim kreira sopstvene sesije za scenarije koje ova ugrađena ne pokriva.

Da bismo demonstrirali ovo u praksi, kreirana je sesija koja hvata isključivo upite čije trajanje prelazi 500 milisekundi, tipičan scenario kada administrator želi obaveštenje o "sporim" upitima bez hvatanja svakog upita na serveru:

```
USE master;
GO

CREATE EVENT SESSION SporiUpiti
ON SERVER
ADD EVENT sqlserver.sql_statement_completed(
    ACTION (sqlserver.sql_text, sqlserver.database_name,
sqlserver.client_hostname)
    WHERE duration > 500000 -- mikrosekunde, 500000 = 500ms
)
ADD TARGET package0.ring_buffer
WITH (MAX_MEMORY = 4096 KB, EVENT_RETENTION_MODE = ALLOW_SINGLE_EVENT_LOSS);
GO

ALTER EVENT SESSION SporiUpiti ON SERVER STATE = START;
```

GO

Nakon pokretanja sesije, poslat je test batch koji sadrži dve naredbe različitog trajanja, kako bi se proverilo da li filter ispravno razdvaja sporo od brzog:

```
USE BankDB;
```

GO

```
WAITFOR DELAY '00:00:01';
```

```
SELECT * FROM Accounts;
```

GO

Podaci uhvaćeni kroz ring_buffer čuvaju se u XML formatu i čitaju se upitom nad sistemskim DMV-ovima koji izlažu sadržaj target-a:

```
SELECT
```

```
    event_data = CAST(target_data AS XML)
```

```
FROM sys.dm_xe_session_targets st
```

```
JOIN sys.dm_xe_sessions s ON s.address = st.event_session_address
```

```
WHERE s.name = 'SporiUpiti';
```

GO

```

event_data1.xml  SQLQuery3.sql...timus (122))*  SQLQuery2.sql...ptimus (55))*  SQLQuery1.sql...ptimus (116))
<RingBufferTarget truncated="0" processingTime="0" totalEventsProcessed="1" eventCount="1" droppedCount="0" memoryUsed="386">
  <event name="sql_statement_completed" package="sqlserver" timestamp="2026-06-29T21:03:06.288Z">
    <data name="duration">
      <type name="int64" package="package0" />
      <value>1013057</value>
    </data>
    <data name="cpu_time">
      <type name="uint64" package="package0" />
      <value>0</value>
    </data>
    <data name="page_server_reads">
      <type name="uint64" package="package0" />
      <value>0</value>
    </data>
    <data name="physical_reads">
      <type name="uint64" package="package0" />
      <value>0</value>
    </data>
    <data name="logical_reads">
      <type name="uint64" package="package0" />
      <value>0</value>
    </data>
    <data name="writes">
      <type name="uint64" package="package0" />
      <value>0</value>
    </data>
    <data name="spills">
      <type name="uint64" package="package0" />
      <value>0</value>
    </data>
    <data name="row_count">
      <type name="uint64" package="package0" />
      <value>0</value>
    </data>
    <data name="last_row_count">
      <type name="uint64" package="package0" />
      <value>0</value>
    </data>
    <data name="line_number">
      <type name="int32" package="package0" />
      <value>2</value>
    </data>
    <data name="offset">
      <type name="int32" package="package0" />
      <value>4</value>
    </data>
    <data name="offset_end">
      <type name="int32" package="package0" />
      <value>56</value>
    </data>
    <data name="statement">
      <type name="unicode_string" package="package0" />
      <value>WAITFOR DELAY '00:00:01';
    </value>
    </data>
    <data name="parameterized_plan_handle">
      <type name="binary_data" package="package0" />
      <value />
    </data>
    <action name="client_hostname" package="sqlserver">
      <type name="unicode_string" package="package0" />
      <value>DESKTOP-AM7N4IV</value>
    </action>
    <action name="database_name" package="sqlserver">
      <type name="unicode_string" package="package0" />
      <value>BankDB</value>
    </action>
    <action name="sql_text" package="sqlserver">
      <type name="unicode_string" package="package0" />
      <value>
        WAITFOR DELAY '00:00:01';
        SELECT * FROM Accounts;
      </value>
    </action>
  </event>
</RingBufferTarget>

```

Slika 4.1. — XML prikazan na slici predstavlja sadržaj ring_buffer target-a nakon izvršavanja test upita. Iako je u istom batch-u poslat i WAITFOR DELAY i SELECT * FROM Accounts, sesija je uhvatila samo jedan događaj, onaj koji odgovara naredbi WAITFOR, čije trajanje (data name="duration") iznosi 1013057 mikrosekundi, odnosno približno 1.01 sekundu. Naredba SELECT nad tabelom Accounts nije se pojavila u rezultatu jer je njeno trajanje bilo znatno kraće od postavljenog praga od 500 milisekundi (WHERE duration > 500000), čime se potvrđuje da predikat na nivou event-a ispravno filtrira događaje pre nego što stignu do target-a.

Pored trajanja, XML sadrži i dodatne action-e definisane prilikom kreiranja sesije, sql_text prikazuje kompletan tekst poslatog batch-a, database_name potvrđuje da je događaj nastao u

bazi BankDB, a `client_hostname` identifikuje radnu stanicu sa koje je upit pokrenut. Ovaj nivo detalja ilustruje osnovnu vrednost Extended Events mehanizma u odnosu na DMV-ove iz prethodnog poglavlja, dok DMV-ovi prikazuju trenutno ili akumulirano stanje, Extended Events beleži pojedinačne, konkretne događaje sa potpunim kontekstom u trenutku kada se dese, što ih čini pogodnim za naknadnu, detaljnu analizu jednog specifičnog problema.

5. Query Store

Query Store rešava problem koji DMV-ovi i Extended Events ne pokrivaju elegantno, istoriju performansi upita kroz vreme. DMV-ovi pokazuju trenutno ili akumulirano stanje od poslednjeg restarta, Extended Events hvataju pojedinačne događaje u realnom vremenu, ali nijedan od ova dva mehanizma sam po sebi ne čuva trajnu istoriju kako se konkretan upit ponašao kroz duži period. Tipičan scenario gde ovo postaje problem je upit koji je mesecima radio brzo i odjednom postane spor, iako se njegov tekst nije promenio. Uzrok je najčešće plan regression, optimizator odluči da za isti upit koristi drugačiji plan izvršavanja, na primer zbog promene statistike ili rasta tabele, i taj novi plan je lošiji od prethodnog. Bez istorije planova, administrator nema način da dokaže šta se promenilo, samo vidi da je upit sada sporiji.

Query Store rešava ovo tako što automatski, u pozadini, snima svaki plan izvršavanja za svaki upit, zajedno sa statistikom performansi (trajanje, CPU vreme, broj izvršavanja). Ovi podaci se čuvaju direktno u bazi, a ne u memoriji kao ring_buffer iz prethodnog poglavlja, pa prežive restart servera. Administrator tako može da uporedi plan od juče sa planom od danas za isti upit i identifikuje tačno kada je došlo do degradacije performansi.

Da bismo demonstrirali Query Store, prvo je uključen na bazi BankDB i podešen da hvata sve upite:

```
ALTER DATABASE BankDB SET QUERY_STORE = ON;  
GO  
ALTER DATABASE BankDB SET QUERY_STORE (  
    OPERATION_MODE = READ_WRITE,  
    QUERY_CAPTURE_MODE = ALL  
);  
  
GO
```

Nakon toga, izvršeno je nekoliko različitih upita nad tabelama Accounts i Transactions, ponovljenih nekoliko puta, kako bi Query Store imao dovoljno podataka za statistiku:

```
USE BankDB;  
GO  
  
SELECT * FROM Accounts;  
SELECT * FROM Transactions WHERE AccountID = 1;  
SELECT a.CustomerName, t.Amount  
FROM Accounts a  
JOIN Transactions t ON a.AccountID = t.AccountID;  
GO
```

Podaci prikupljeni kroz Query Store dostupni su preko sistemskih pogleda sys.query_store_query, sys.query_store_query_text, sys.query_store_plan i sys.query_store_runtime_stats, koji povezani zajedno daju tekst upita, plan izvršavanja i statistiku performansi za svaki snimljeni upit.

Pri prvom upitu nad ovim pogledima u rezultatu su se pojavili i upiti koje SSMS interno šalje radi IntelliSense funkcionalnosti (auto-complete, popunjavanje liste tabela i kolona), pa je upit dodatno filtriran kako bi prikazao isključivo upite relevantne za demonstraciju:

SQLQuery1.s...timus (132)*

```

1  SELECT TOP 10
2      qt.query_sql_text,
3      rs.avg_duration,
4      rs.avg_cpu_time,
5      rs.count_executions,
6      rs.last_execution_time
7  FROM sys.query_store_query q
8  JOIN sys.query_store_query_text qt ON q.query_text_id = qt.query_text_id
9  JOIN sys.query_store_plan p ON q.query_id = p.query_id
10 JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
11 WHERE qt.query_sql_text LIKE '%Accounts%'
12       OR qt.query_sql_text LIKE '%Transactions%'
13 ORDER BY rs.avg_duration DESC;
14 GO

```

110 % No issues found

Results Messages

	query_sql_text	avg_duration	avg_cpu_time	count_executions	last_execution_time
1	SELECT * FROM Accounts	46.5	46.3333333333333	6	2026-06-30 18:17:59.7200000 +00:00
2	SELECT a.CustomerName,t.Amount FROM Accounts...	33.5	33.1666666666667	6	2026-06-30 18:17:59.7200000 +00:00
3	(@1 tinyint)SELECT * FROM [Transactions] WHERE [A...	15.5	15.5	6	2026-06-30 18:17:59.7200000 +00:00

Slika 5.1. — rezultat prikazan potvrđuje da je Query Store ispravno prepoznao i razdvojio tri različita upita izvršena nad bazom, *SELECT * FROM Accounts*, JOIN upit nad *Accounts* i *Transactions*, i parametrizovani *SELECT* nad tabelom *Transactions*. Kolona *count_executions* pokazuje vrednost 6 za sva tri upita, što potvrđuje da je Query Store akumulirao statistiku kroz više ponovljenih izvršavanja, a ne samo poslednje. Kolona *avg_duration* pokazuje da je *SELECT * FROM Accounts* u proseku trajao 46.5 mikrosekundi, dok je upit nad *Transactions* sa *WHERE* klauzulom bio najbrži (15.5 mikrosekundi), što je očekivano s obzirom na manju količinu podataka koju filtrirani upit obrađuje u odnosu na nefiltrirani *SELECT*.

Ono što u ovoj demonstraciji nije prikazano, ali predstavlja ključnu vrednost Query Store-a u produkcionom okruženju, jeste istorija različitih planova izvršavanja za isti upit. Pogled *sys.query_store_plan* može za jedan *query_id* sadržati više različitih *plan_id* vrednosti, svaki sa sopstvenom statistikom trajanja i CPU vremena. Kada administrator utvrdi da je noviji plan lošiji od prethodnog, može eksplicitno naložiti optimizatoru da koristi stariji, dokazano bolji plan, naredbom:

```
EXEC sp_query_store_force_plan @query_id = ..., @plan_id = ...;
```

Time se degradacija performansi može ispraviti bez izmene samog SQL koda aplikacije, što je posebno korisno u sistemima gde promena aplikativnog sloja zahteva ceo release ciklus, dok se plan forsira trenutno.

Ovaj primer ilustruje osnovnu razliku Query Store-a u odnosu na prethodna dva mehanizma, dok DMV-ovi i Extended Events prikazuju pojedinačne snimke ili događaje, Query Store agregira performanse istog upita kroz vreme, omogućavajući administratoru da prati trend, a ne samo jedno izvršavanje.

6. Proceduralni deo, zašto se monitoring pakuje u proceduru?

Mehanizmi predstavljeni u prethodnim poglavljima, DMV-ovi, Extended Events i Query Store, daju administratoru sve potrebne podatke, ali zahtevaju da se upiti ručno pišu i pokreću svaki put kada administrator želi da proveri stanje servera. U produkcionim okruženjima ovo nije održivo, administrator ne može da bude prisutan u svakom trenutku i ručno pokreće desetine upita kad god posumnja na problem, a baš ti problemi (blokade, rast baze, neuobičajeni wait-ovi) često nastaju van radnog vremena, kada niko ne gleda ekran.

Standardna praksa u radu sa SQL Serverom je da se najvažniji dijagnostički upiti objedine u jednu stored proceduru, koja se zatim može pozvati ručno, u bilo kom trenutku, ili automatski, kroz SQL Server Agent job koji se izvršava po rasporedu (na primer na svakih 15 minuta). Procedura tipično vraća sažet pregled stanja sistema, najznačajnija čekanja, broj trenutno aktivnih i blokiranih sesija, veličinu baze, i slično, čime se administratoru daje "brz pregled na prvi pogled", bez potrebe da pamti i ručno kuca svaki put isti niz upita.

Pored praktičnosti, ovaj pristup ima i prednost konzistentnosti, svaki put kada se procedura pozove, izvršava se identičan skup provera, na isti način, što olakšava poređenje stanja sistema kroz vreme i smanjuje rizik da administrator u žurbi zaboravi da proveri neki važan pokazatelj. Kada se procedura dodatno poveže sa tabelom za logovanje i SQL Server Agent jobom, dobija se kontinuirano, automatizovano praćenje stanja baze, bez potrebe za ručnom intervencijom, što predstavlja prirodan sledeći korak nakon pojedinačnih dijagnostičkih upita predstavljenih u prethodnim poglavljima.

Procedura je kreirana naredbom CREATE OR ALTER, što omogućava da se bez greške ponovo izvrši i ažurira ukoliko se logika kasnije proširi, bez potrebe da se prvo eksplicitno briše.

```
USE BankDB;
GO

CREATE OR ALTER PROCEDURE sp_BankDB_HealthCheck
AS
BEGIN
    SET NOCOUNT ON;

    -- 1. Top 5 wait tipova (filtrirano)
    SELECT TOP 5
        wait_type,
        wait_time_ms,
        waiting_tasks_count
    FROM sys.dm_os_wait_stats
    WHERE wait_type NOT IN (
        'SOS_WORK_DISPATCHER', 'DISPATCHER_QUEUE_SEMAPHORE', 'SLEEP_TASK',
        'LOGMGR_QUEUE', 'LAZYWRITER_SLEEP', 'DIRTY_PAGE_POLL',
        'PWAIT_DIRECTLOGCONSUMER_GETNEXT', 'HADR_FILESTREAM_IOMGR_IOCOMPLETION',
        'QDS_PERSIST_TASK_MAIN_LOOP_SLEEP'
    )
    ORDER BY wait_time_ms DESC;
```



```

-- 2. Trenutno aktivne i blokirane sesije
SELECT
    session_id,
    status,
    blocking_session_id,
    wait_type,
    command
FROM sys.dm_exec_requests
WHERE session_id != @@SPID;

-- 3. Velicina baze
SELECT
    DB_NAME() AS database_name,
    SUM(size * 8 / 1024) AS size_mb
FROM sys.database_files;
END;
GO

```

Implementirana procedura sp_BankDB_HealthCheck objedinjuje tri dijagnostička upita u jedan poziv, top wait statistike (princip identičan onom iz poglavlja 3.2), pregled trenutno aktivnih i potencijalno blokiranih sesija (princip iz poglavlja 3.1, proširen kolonom blocking_session_id), i trenutnu veličinu baze podataka.

SQLQuery1.s...timus (132))*

1

EXEC sp_BankDB_HealthCheck;

2

GO

110 %

No issues found

Results

Messages

	wait_type	wait_time_ms	waiting_tasks_count
1	QDS_ASYNC_QUEUE	1381129062	10
2	PREEMPTIVE_XE_DISPATCHER	105166000	3
3	CHECKPOINT_QUEUE	97779586	11
4	XE_DISPATCHER_WAIT	80504730	340
5	SP_SERVER_DIAGNOSTICS_SLEEP	80462620	3293

	session_id	status	blocking_session_id	wait_type	command
1	1	sleeping	0	NULL	TASK MANAGER
2	2	sleeping	0	NULL	TASK MANAGER
3	3	sleeping	0	NULL	TASK MANAGER
4	4	sleeping	0	NULL	TASK MANAGER
5	5	sleeping	0	NULL	TASK MANAGER
6	6	sleeping	0	NULL	TASK MANAGER
7	7	sleeping	0	NULL	TASK MANAGER
8	8	sleeping	0	NULL	TASK MANAGER
9	9	sleeping	0	NULL	TASK MANAGER
10	10	sleeping	0	NULL	TASK MANAGER
11	11	background	0	BROKER_TASK_STOP	UCS_TASK_NO_SPEC
12	12	background	0	BROKER_TASK_STOP	UCS_TASK_NO_SPEC
13	13	sleeping	0	NULL	TASK MANAGER
14	14	background	0	BROKER_TASK_STOP	UCS_TASK_NO_SPEC

	database_name	size_mb
1	BankDB	16

Slika 6.1. — rezultat poziva sastoji se iz tri odvojena seta podataka, u skladu sa tri *SELECT* naredbe unutar procedure. Prvi set prikazuje top 5 wait tipova, gde se, slično kao u poglavlju 3.2, ponovo izdvajaju isključivo sistemski, benigni tipovi čekanja (*QDS_ASYNC_QUEUE* vezan za Query Store, *CHECKPOINT_QUEUE* i *XE_DISPATCHER_WAIT* vezani za Extended Events), što potvrđuje da server u trenutku poziva nije bio pod pritiskom. Treba napomenuti da filter lista unutar procedure namerno nije identična onoj iz poglavlja 3.2, već je svedena na najčešće tipove radi preglednosti koda, što znači da se povremeno mogu pojaviti dodatni benigni tipovi koji nisu eksplicitno isključeni, administrator ih i dalje prepoznaje na osnovu imena kao sistemске, a ne kao signal problema.

Drugi set prikazuje listu trenutno aktivnih sesija, gde kolona `blocking_session_id` za sve redove ima vrednost 0, što znači da u trenutku poziva nije postojala nijedna blokirana sesija. Ovaj rezultat predstavlja "čisto" stanje sistema, koje će biti namerno narušeno u poglavlju 7, kako bi se demonstrovalo ponašanje iste procedure u prisustvu stvarne blokade. Treći set potvrđuje da baza BankDB trenutno zauzima 16 MB. Ovaj primer ilustruje praktičnu vrednost objedinjavanja dijagnostičkih upita u proceduru, umesto tri odvojena upita koja bi administrator morao ručno da pokrene i poveže u celovitu sliku, jedan poziv `EXEC sp_BankDB_HealthCheck` daje kompletan presek stanja sistema.

Prethodna verzija procedure vraća rezultate kao tri odvojena result set-a, što je pogodno za ručno pokretanje iz SSMS-a. Za automatizovano izvršavanje kroz SQL Server Agent, prirodniji pristup je da procedura zapisuje aggregate u namensku tabelu za istoriju, čime se dobija vremenska serija koja se kasnije može analizirati. Za tu svrhu kreirana je tabela `HealthCheckLog` i modifikovana verzija procedure koja ne vraća rezultate klijentu, već ih upisuje u tabelu:

```
CREATE TABLE HealthCheckLog (
    LogID INT IDENTITY PRIMARY KEY,
    RunTime DATETIME2 DEFAULT SYSUTCDATETIME(),
    TopWaitType NVARCHAR(60),
    TopWaitTimeMs BIGINT,
    ActiveSessions INT,
    BlockedSessions INT,
    DatabaseSizeMB INT
);
GO

CREATE OR ALTER PROCEDURE sp_BankDB_HealthCheckLog
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @topWaitType NVARCHAR(60),
            @topWaitTimeMs BIGINT,
            @activeSessions INT,
            @blockedSessions INT,
            @databaseSizeMB INT;

    SELECT TOP 1
        @topWaitType = wait_type,
        @topWaitTimeMs = wait_time_ms
    FROM sys.dm_os_wait_stats
    WHERE wait_type NOT IN (
        'SOS_WORK_DISPATCHER', 'DISPATCHER_QUEUE_SEMAPHORE', 'SLEEP_TASK',
        'LOGMGR_QUEUE', 'LAZYWRITER_SLEEP', 'DIRTY_PAGE_POLL',

        'PWAIT_DIRECTLOGCONSUMER_GETNEXT', 'HADR_FILESTREAM_IOMGR_IOCOMPLETION',
        'QDS_PERSIST_TASK_MAIN_LOOP_SLEEP'
    )
    ORDER BY wait_time_ms DESC;

    SELECT
        @activeSessions = COUNT(*),
```

```

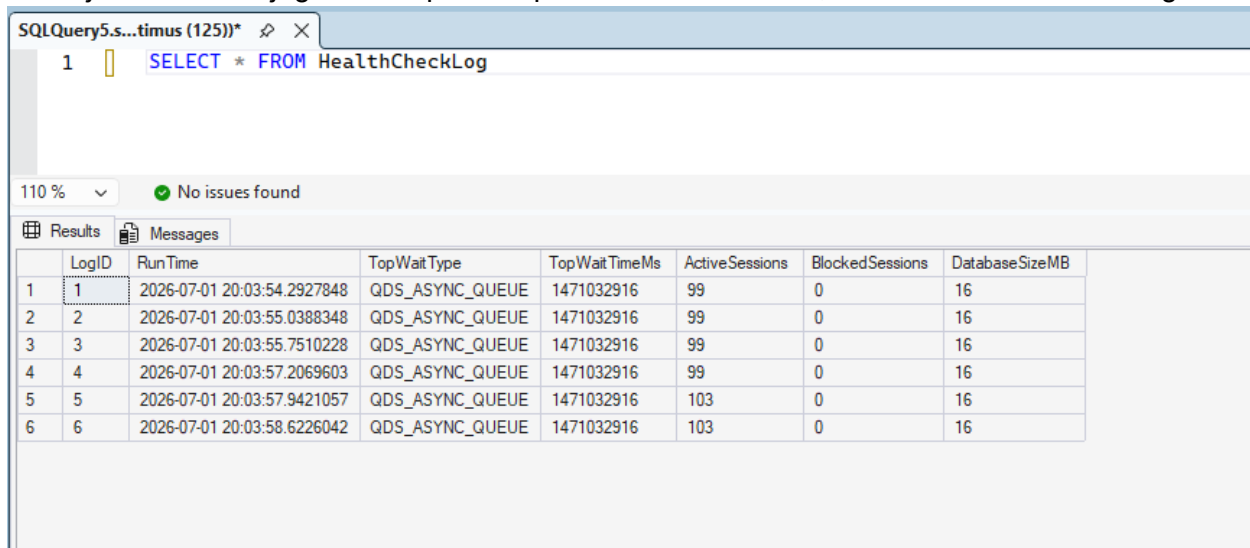
        @blockedSessions = SUM(CASE WHEN blocking_session_id != 0 THEN 1
ELSE 0 END)
FROM sys.dm_exec_requests
WHERE session_id != @@SPID;

SELECT @databaseSizeMB = SUM(size * 8 / 1024)
FROM sys.database_files;

INSERT INTO HealthCheckLog
(TopWaitType, TopWaitTimeMs, ActiveSessions, BlockedSessions,
DatabaseSizeMB)
VALUES
(@topWaitType, @topWaitTimeMs, @activeSessions, @blockedSessions,
@databaseSizeMB);
END;
GO

```

Kada se ovako definisana procedura poveže sa SQL Server Agent job-om koji je izvršava u pravilnim intervalima (na primer svakih 15 minuta), dobija se vremenska serija u tabeli HealthCheckLog na osnovu koje administrator može da uporedi opterećenje sistema tokom radnog vremena i noću, ili da detektuje periode kada broj blokada raste iznad normalnog. Time se od jednokratne dijagnostičke provere prelazi na kontinuirani, automatizovani monitoring.



LogID	RunTime	TopWaitType	TopWaitTimeMs	ActiveSessions	BlockedSessions	DatabaseSizeMB
1	2026-07-01 20:03:54.2927848	QDS_ASYNC_QUEUE	1471032916	99	0	16
2	2026-07-01 20:03:55.0388348	QDS_ASYNC_QUEUE	1471032916	99	0	16
3	2026-07-01 20:03:55.7510228	QDS_ASYNC_QUEUE	1471032916	99	0	16
4	2026-07-01 20:03:57.2069603	QDS_ASYNC_QUEUE	1471032916	99	0	16
5	2026-07-01 20:03:57.9421057	QDS_ASYNC_QUEUE	1471032916	103	0	16
6	2026-07-01 20:03:58.6226042	QDS_ASYNC_QUEUE	1471032916	103	0	16

Slika 6.2. — Nakon šest uzastopnih poziva procedure `sp_BankDB_HealthCheckLog`, tabela `HealthCheckLog` sadrži šest redova, svaki sa jedinstvenom `RunTime` vrednošću, a preostale kolone (`TopWaitType`, `ActiveSessions`, `BlockedSessions`, `DatabaseSizeMB`) pokazuju stanje sistema u trenutku svakog snimanja. Kolona `BlockedSessions` ima vrednost 0 u svim redovima, što potvrđuje da tokom snimanja nije bilo aktivnih blokada, dok se u koloni `ActiveSessions` vidi blaga promena vrednosti sa 99 na 103 između četvrtog i petog poziva, što ilustruje kako ovakva vremenska serija hvata i manje varijacije opterećenja koje jedan snimak stanja ne bi otkrio. Ovakva istorija predstavlja osnovu za analizu trendova opterećenja sistema kroz vreme, i direktan je preduslov za povezivanje procedure sa SQL Server Agent job-om, koji bi je izvršavao u pravilnim intervalima bez potrebe za ručnom intervencijom administratora.

7. Blocking i deadlock

Blocking i deadlock su dva međusobno povezana, ali suštinski različita fenomena koja nastaju kada se konkurentne transakcije nadmeću za isti resurs, najčešće red ili stranicu podataka u tabeli.

Blocking nastaje kada jedna transakcija drži lock nad resursom, na primer zbog UPDATE-a koji još nije commit-ovan, a druga transakcija pokušava da pristupi istom resursu i mora da sačeka dok prva ne završi i otpusti lock. Blocking sam po sebi nije greška, već je normalan i očekivan deo rada SQL Servera, mehanizam zaključavanja postoji upravo da garantuje da se transakcije ne mešaju i ne narušavaju konzistentnost podataka. Problem nastaje kada blocking traje predugo, jer to korisnici percipiraju kao da je sistem zapeo, iako server tehnički normalno radi, samo čeka. Deadlock je poseban, ozbiljniji slučaj blockinga, do njega dolazi kada dve ili više transakcija čekaju jedna na drugu u krugu, transakcija A drži resurs koji čeka transakcija B, dok istovremeno transakcija B drži resurs koji čeka transakcija A. SQL Server u ovoj situaciji koristi ugrađen mehanizam, deadlock monitor, koji periodično proverava da li postoji ovakav ciklus čekanja i, kada ga pronađe, bira jednu od transakcija kao žrtvu, prekida je i vraća joj grešku, čime se ciklus razbija. Ključna razlika za administratora je u tome što se blocking rešava sam, čim transakcija koja drži resurs završi, dok deadlock SQL Server mora aktivno da razreši žrtvovanjem jedne strane, jer bi inače sistem ostao zaglavljen zauvek.

Da bismo demonstrirali blocking u praksi, simulirana je situacija sa dve konkurentne transakcije nad istim redom tabele Accounts. U prvoj sesiji pokrenuta je eksplicitna transakcija koja menja stanje računa sa AccountID = 1, ali namerno ne izvršava COMMIT odmah, već zadržava transakciju otvorenu jedan minut:

```
USE BankDB;
GO
BEGIN TRANSACTION;
UPDATE Accounts
SET Balance = Balance - 100
WHERE AccountID = 1;
WAITFOR DELAY '00:01:00';
COMMIT TRANSACTION;
```

Dok je prva transakcija aktivna i drži lock nad redom, u drugoj sesiji pokrenuta je naredba koja pokušava da izmeni isti red:

```
USE BankDB;
GO
UPDATE Accounts
SET Balance = Balance - 50

WHERE AccountID = 1;
```

Ova naredba ostaje u stanju izvršavanja i ne vraća rezultat, jer čeka da prva transakcija otpusti lock. U trećoj sesiji, dok je druga sesija u ovom stanju čekanja, izvršen je upit koji eksplicitno filtrira isključivo sesije koje su trenutno blokirane:

```

1  SELECT
2      session_id,
3      status,
4      blocking_session_id,
5      wait_type,
6      command
7  FROM sys.dm_exec_requests
8  WHERE blocking_session_id != 0;
9  GO

```

10 % No issues found

Results Messages

	session_id	status	blocking_session_id	wait_type	command
1	74	suspended	132	LCK_M_X	UPDATE

Slika 7.1. — rezultat potvrđuje blocking u realnom vremenu. Sesija 74, koja odgovara drugoj UPDATE naredbi, ima status *suspended*, što znači da aktivno čeka i ne koristi CPU. Kolona *blocking_session_id* pokazuje vrednost 132, što je tačno *session_id* prve sesije koja drži otvorenu transakciju, čime se eksplicitno potvrđuje koja sesija blokira koju. Kolona *wait_type* prikazuje vrednost *LCK_M_X*, što označava čekanje na *exclusive lock*, tip zaključavanja koji UPDATE naredba zahteva nad redom koji menja, a koji ostaje aktivan sve dok transakcija koja ga drži ne izvrši COMMIT ili ROLLBACK. Nakon isteka minuta i izvršavanja COMMIT TRANSACTION u prvoj sesiji, lock se automatski oslobađa i druga UPDATE naredba odmah nastavlja izvršavanje, čime se potvrđuje da je blocking, za razliku od deadlocka, situacija koja se sama razrešava čim resurs postane slobodan, bez potrebe za intervencijom SQL Servera ili administratora.

Za još detaljniju sliku toga šta je tačno zaključano, može se konsultovati pogled *sys.dm_tran_locks* pomenut u poglavlju 3.1, koji izlaže svaki pojedinačni lock na serveru:

SQLQuery9.s...timus (123))* X SQLQuery8.sql... Executing...* SQLQuery7.sql... Executing...*

```
1 SELECT
2     request_session_id,
3     resource_type,
4     resource_description,
5     request_mode,
6     request_status
7 FROM sys.dm_tran_locks
8 WHERE resource_type IN ('KEY', 'PAGE', 'OBJECT', 'RID')
9     AND request_session_id != @@SPID
10 ORDER BY request_session_id, resource_type;
```

110 % No issues found

Results Messages

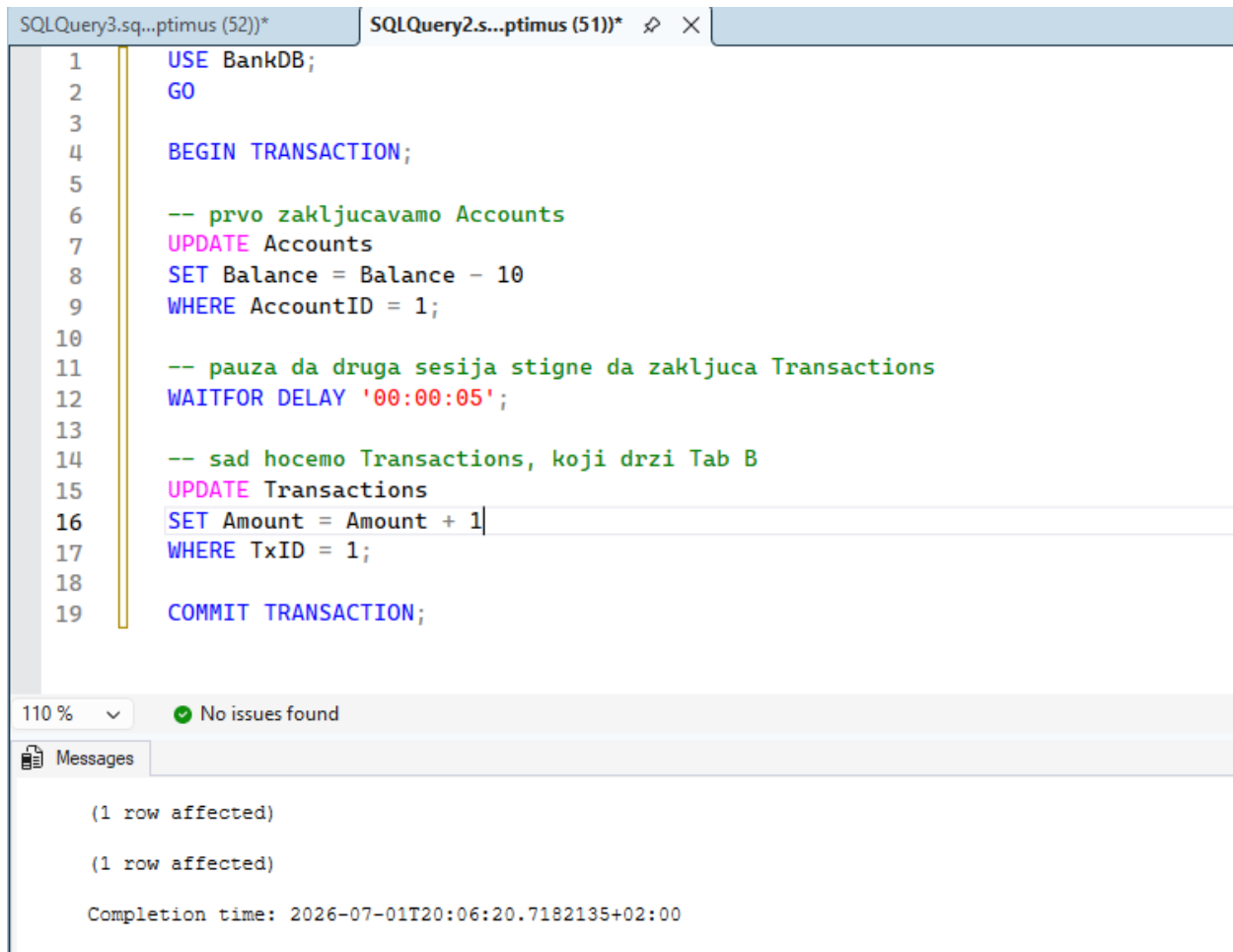
	request_session_id	resource_type	resource_description	request_mode	request_status
1	121	KEY	(8194443284a0)	X	GRANT
2	121	OBJECT		IX	GRANT
3	121	PAGE	1:536	IX	GRANT
4	122	KEY	(8194443284a0)	X	WAIT
5	122	OBJECT		IX	GRANT
6	122	PAGE	1:536	IX	GRANT
7	124	PAGE	1:617	IX	GRANT
8	124	PAGE	1:302	IX	GRANT
9	124	PAGE	1:303	IX	GRANT
10	124	PAGE	1:408	IX	GRANT
11	124	PAGE	1:423	IX	GRANT
12	124	PAGE	1:424	IX	GRANT
13	124	RID	1:617:2	X	GRANT

Slika 7.2. — rezultat prikazuje istovremeni presek zaključavanja u toku blokade iz Slike 7.1. Sesija 121 drži exclusive lock (*request_mode* = X) nad ključem tabele *Accounts* sa statusom GRANT, dok sesija 122 čeka exclusive lock nad istim resursom (identičan *resource_description*) sa statusom WAIT, u ovoj vezi jasno je vidljivo koja sesija blokira koju i oko koje se tačke sistema nadmeću. Pored ključa, sesija 121 drži i intent exclusive (IX) lock-ove na višem nivou hijerarhije, na stranici (PAGE) i objektu (OBJECT), što je standardan mehanizam SQL Servera kojim se signalizira da unutar tog većeg resursa postoji finije zaključavanje na nivou pojedinačnog reda. Sesija 124, vidljiva u ostatku rezultata, predstavlja nezavisnu aktivnost druge sesije na serveru i ne učestvuje u prikazanoj blokadi. Ova granularnost, koju *sys.dm_exec_requests* iz Slike 7.1 ne izlaže, dozvoljava administratoru da tačno identifikuje resurs oko kojeg se sesije nadmeću.

Za razliku od blockinga, deadlock zahteva specifičan raspored operacija između najmanje dve sesije koje pristupaju istim resursima obrnutim redosledom. Pre nego što se demonstrira deadlock, korisno je pomenuti kako SQL Server perzistira informacije o njemu. Po default-u, svaki deadlock se automatski upisuje u ugrađenu Extended Events sesiju *system_health* (predstavljenu u poglavlju 4), gde je dostupan u obliku *xml_deadlock_report* event-a, koji sadrži kompletan deadlock graph — obe strane, resurse i redosled operacija koje su dovele do ciklusa. Alternativno, administrator može uključiti trace flag 1222 naredbom *DBCC TRACEON (1222, -1)*, čime se ista informacija dodatno upisuje u error log servera u čitljivom tekstualnom obliku. U ovoj demonstraciji fokus je na neposrednoj klijentskoj perspektivi, tj. na poruci Msg 1205 koju SQL Server šalje aplikaciji, dok se detaljna analiza deadlock graph-a iz *system_health* sesije ostavlja kao prirodan naredni korak u istraživanju konkretnih deadlock-ova u produkciji:

```
DBCC TRACEON (1222, -1);  
GO
```

U prvoj sesiji pokrenuta je transakcija koja prvo zaključava red u tabeli Accounts, zatim pravi kratku pauzu kako bi druga sesija imala vremena da zaključa svoj resurs, i tek onda pokušava da pristupi tabeli Transactions:



The screenshot displays the SQL Server Enterprise Manager interface. At the top, two query windows are open: 'SQLQuery3.sq...ptimus (52))*' and 'SQLQuery2.s...ptimus (51))*'. The first window contains a SQL script with 19 lines of code. The script starts with 'USE BankDB;' and 'GO'. It then begins a transaction with 'BEGIN TRANSACTION;'. Line 6 is a comment: '-- prvo zakljucavamo Accounts'. Line 7 is 'UPDATE Accounts', line 8 is 'SET Balance = Balance - 10', and line 9 is 'WHERE AccountID = 1;'. Line 11 is a comment: '-- pauza da druga sesija stigne da zakljuca Transactions'. Line 12 is 'WAITFOR DELAY '00:00:05';'. Line 14 is a comment: '-- sad hocemo Transactions, koji drzi Tab B'. Line 15 is 'UPDATE Transactions', line 16 is 'SET Amount = Amount + 1', line 17 is 'WHERE TxID = 1;', and line 19 is 'COMMIT TRANSACTION;'. Below the query window, there is a status bar showing '110 %' and 'No issues found'. At the bottom, a 'Messages' window is open, showing two messages: '(1 row affected)' and '(1 row affected)'. The completion time is displayed as '2026-07-01T20:06:20.7182135+02:00'.

```
1 USE BankDB;  
2 GO  
3  
4 BEGIN TRANSACTION;  
5  
6 -- prvo zakljucavamo Accounts  
7 UPDATE Accounts  
8 SET Balance = Balance - 10  
9 WHERE AccountID = 1;  
10  
11 -- pauza da druga sesija stigne da zakljuca Transactions  
12 WAITFOR DELAY '00:00:05';  
13  
14 -- sad hocemo Transactions, koji drzi Tab B  
15 UPDATE Transactions  
16 SET Amount = Amount + 1  
17 WHERE TxID = 1;  
18  
19 COMMIT TRANSACTION;
```

110 % No issues found

Messages

(1 row affected)

(1 row affected)

Completion time: 2026-07-01T20:06:20.7182135+02:00

U drugoj sesiji, pokrenutoj u toku pauze prve sesije, zaključavanje se obavlja obrnutim redosledom, prvo Transactions, zatim Accounts:


```
SQLQuery3.s...ptimus (52))" x SQLQuery2.sq...ptimus (51))"
1  USE BankDB;
2  GO
3
4  BEGIN TRANSACTION;
5
6  -- prvo zakljucavamo Transactions
7  UPDATE Transactions
8  SET Amount = Amount + 1
9  WHERE TxID = 1;
10
11 -- pauza da prva sesija stigne da zakljuca Accounts
12 WAITFOR DELAY '00:00:05';
13
14 -- sad hocemo Accounts, koji drzi Tab A
15 UPDATE Accounts
16 SET Balance = Balance - 20
17 WHERE AccountID = 1;
18
19 COMMIT TRANSACTION;
```

110 % No issues found

Messages

(1 row affected)
Msg 1205, Level 13, State 51, Line 15
Transaction (Process ID 52) was deadlocked on lock resources with another process and has been chosen as the deadlock victim. Rerun the transaction.
Completion time: 2026-07-01T20:06:20.7234952+02:00

Slika 7.3. — rezultat prikazuje ponašanje sesije koja je odabrana kao deadlock victim (Process ID 52). Nakon što je prva UPDATE naredba u ovoj sesiji uspešno izvršena i vraćena je poruka "(1 row affected)", druga UPDATE naredba prekinuta je greškom Msg 1205 sa porukom "Transaction was deadlocked on lock resources with another process and has been chosen as the deadlock victim. Rerun the transaction." Sesija koja je odabrana kao žrtva automatski je rollback-ovana, a njena promena stanja računa nije trajno primenjena na bazu. Istovremeno, druga sesija je nesmetano nastavila izvršavanje i uspešno commit-ovala svoju transakciju, čime se potvrđuje da SQL Server, za razliku od blockinga koji se razrešava strpljenjem, aktivno razrešava deadlock žrtvovanjem jedne od strana kako bi ostala mogla da napreduje.

U produkcionim aplikacijama ovo ponašanje ima direktnu praktičnu implikaciju, aplikacija mora biti napisana da očekuje grešku 1205 i da automatski ponovi transakciju (retry logika), jer je deadlock nepredvidiva situacija koja može da se javi u bilo kom trenutku pri visokoj konkurenciji, čak i uz besprekorno napisan kod.

8. Zaključak

Ovaj rad prikazao je implementaciju monitoringa i observabilnosti u Microsoft SQL Serveru kroz teorijsku osnovu i praktične demonstracije na primeru baze podataka BankDB. Kroz poglavlja rada demonstrirano je kako SQL Server, kroz kombinaciju ugrađenih mehanizama, pokriva tri centralne kategorije monitoringa obrađene u ovom radu: aktivne sesije i akumulirana čekanja, performanse pojedinačnih upita kroz vreme, i konkurentne transakcije sa fokusom na blokade i deadlock-ove. Dynamic Management Views omogućavaju uvid u trenutno stanje servera i identifikaciju aktivnih sesija, dok wait statistics pružaju dugoročnu sliku akumuliranih čekanja i usmeravaju administratora ka uzroku problema. Extended Events, kao naslednik starijeg SQL Trace mehanizma, omogućava precizno snimanje pojedinačnih događaja uz minimalan uticaj na performanse servera, sa filterima koji hvataju isključivo relevantne događaje. Query Store dopunjuje ove mehanizme trajnom istorijom planova izvršavanja i statistike upita, čineći vidljivim degradacije performansi koje bi bez ovog alata ostale neobjašnjive. Proceduralni deo rada pokazao je kako se dijagnostička logika zasnovana na DMV-ovima može upakovati u stored proceduru `sp_BankDB_HealthCheck`, koja u jednom pozivu daje kompletan presek stanja sistema. Ovaj pristup, standardan u produkcionim okruženjima, smanjuje zavisnost od ručnog pokretanja upita i otvara mogućnost za automatizaciju kroz SQL Server Agent jobove. Poglavlje o blockingu i deadlocku demonstriralo je kako konkurentne transakcije u bankarskom sistemu mogu dovesti do blokade, i kako se ona detektuje u realnom vremenu kroz DMV-ove, čime se zatvara krug između teorijskih osnova iz poglavlja 2 i praktičnih alata predstavljenih u ostatku rada.

Monitoring nije jednokratno podešavanje već kontinuiran proces, dijagnostičke upite treba redovno pokretati, wait statistiku periodično resetovati i analizirati trendove, a nove dijagnostičke potrebe ugrađivati u postojeće procedure. Kombinacija mehanizama predstavljenih u ovom radu predstavlja solidan temelj za proaktivno upravljanje performansama i dostupnošću u savremenim bazama podataka.

9. Literatura

1. Microsoft, "Dynamic Management Views and Functions (Transact-SQL)", Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/sql/relational-databases/system-dynamic-management-views/system-dynamic-management-views?view=sql-server-ver17>
2. Microsoft, "sys.dm_exec_requests (Transact-SQL)", Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/sql/relational-databases/system-dynamic-management-views/sys-dm-exec-requests-transact-sql?view=sql-server-ver17>
3. Microsoft, "sys.dm_os_wait_stats (Transact-SQL)", Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/sql/relational-databases/system-dynamic-management-views/sys-dm-os-wait-stats-transact-sql?view=sql-server-ver17>
4. Microsoft, "Extended Events Overview", Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/sql/relational-databases/extended-events/extended-events?view=sql-server-ver17>
5. Microsoft, "Monitoring Performance by Using the Query Store", Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/sql/relational-databases/performance/monitoring-performance-by-using-the-query-store?view=sql-server-ver17>
6. Microsoft, "CREATE PROCEDURE (Transact-SQL)", Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/sql/t-sql/statements/create-procedure-transact-sql?view=sql-server-ver17>
7. Microsoft, "Detect and End Deadlocks", Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/sql/relational-databases/sql-server-deadlocks-guide?view=sql-server-ver17>
8. Microsoft, "Transaction Locking and Row Versioning Guide", Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/sql/relational-databases/sql-server-transaction-locking-and-row-versioning-guide?view=sql-server-ver17>